PROJECT DOCUMENTATION: NOTEPAD APPLICATION

1. Project Overview

The Notepad Application is a very light weight application with a good range of features. It's built using Java Swing, Apache NetBeans and Apache Maven. It's a team project so it has all the standard notepad functionality plus a number of extra features to boost productivity such as custom file filters, font formatting, a whole host of keyboard shortcuts, a dynamic status bar and unit tests that run automatically.

2. System Features & Core Modules

The application is divided into five main functional modules, each implemented by a team member:

Module 1: File I/O Operations

- **Open File (openFile)**: limits the file chooser to just .txt files. Opens the selected file via a BufferedReader and loads it into the main text area, updating the window title in the process.
- **Save File As (`saveFileAs`)**: prompts the user for a save location, adds a .txt extension if one isn't already present and saves the file.
- **New File (`newFile`)**: Checks to see if there's any unsaved text in the text area. If there is, it will prompt the user to save before it clears the workspace and resets the window title to "Untitled".

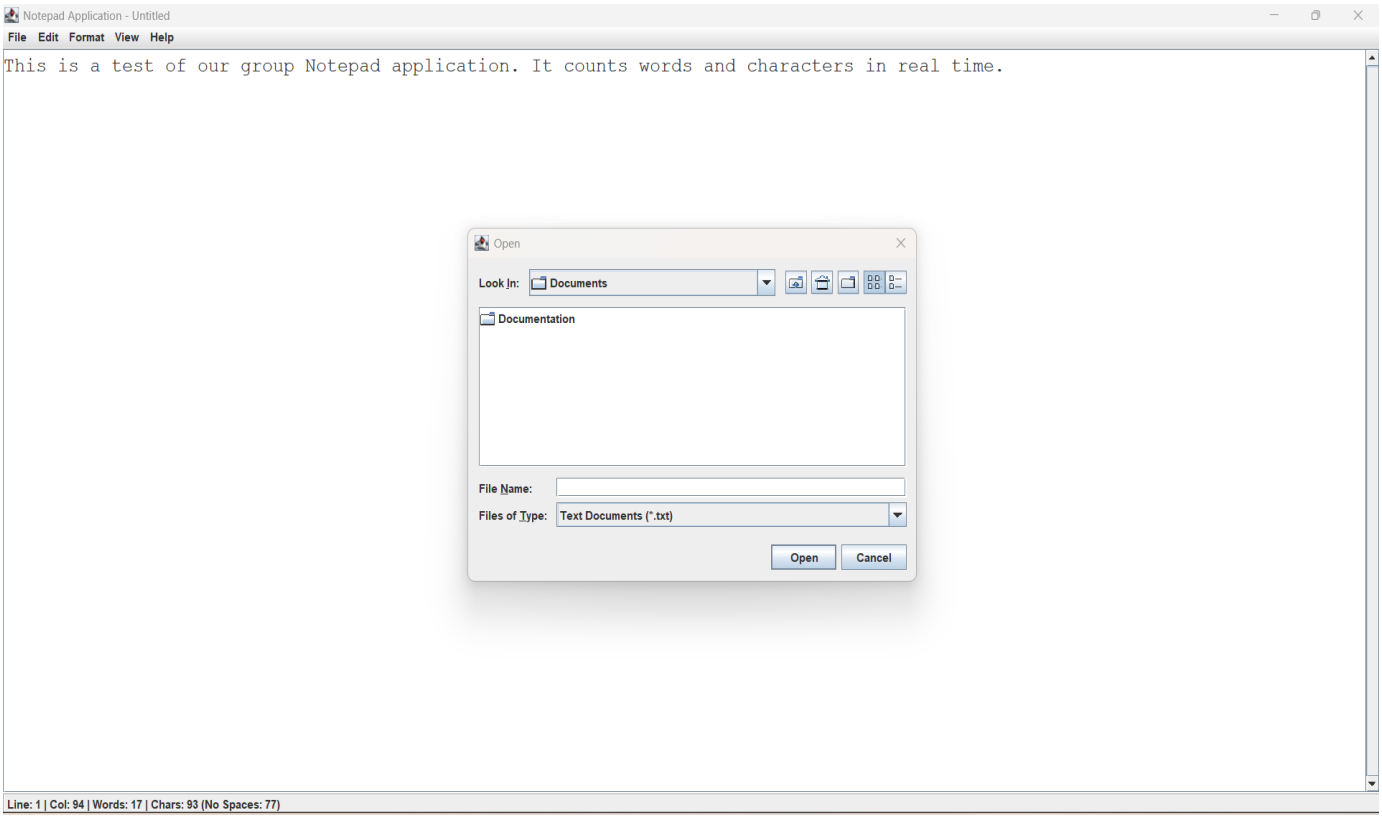


Figure 1: The File Open/ Save Dialog

This screenshot shows the File Open dialog opened from the File menu. The user can browse and select a text file to open in the Notepad application. The Save dialog works in the same way and allows the user to save the document. This proves that the File I/O feature described in Module 1 is working correctly.

Module 2: Edit Actions & Shortcuts (Ndumiso Mthembu)

- **Keyboard Accelerators**: Bind the standard shortcut keys (Cut = Ctrl+X, Copy = Ctrl+C, Paste = Ctrl+V, Select All = Ctrl+a).
- **A 'Clear All' option**: adds a "Clear All" menu item complete with a confirmation dialog (via JOptionPane) to prevent accidental loss of data should the user wish to clear the text area.

Module 3: Font Selection & Formatting

- **Custom Font Dialog (FontChooserDialog)**: looks at the available system graphics environments to get the system fonts. The user is then prompted to select a font family, style and size via a custom dialog.
- **Font size update (`changeFontSize`)**: The font chooser dialog is now integrated with the main text area so that the active text font is changed as appropriate.

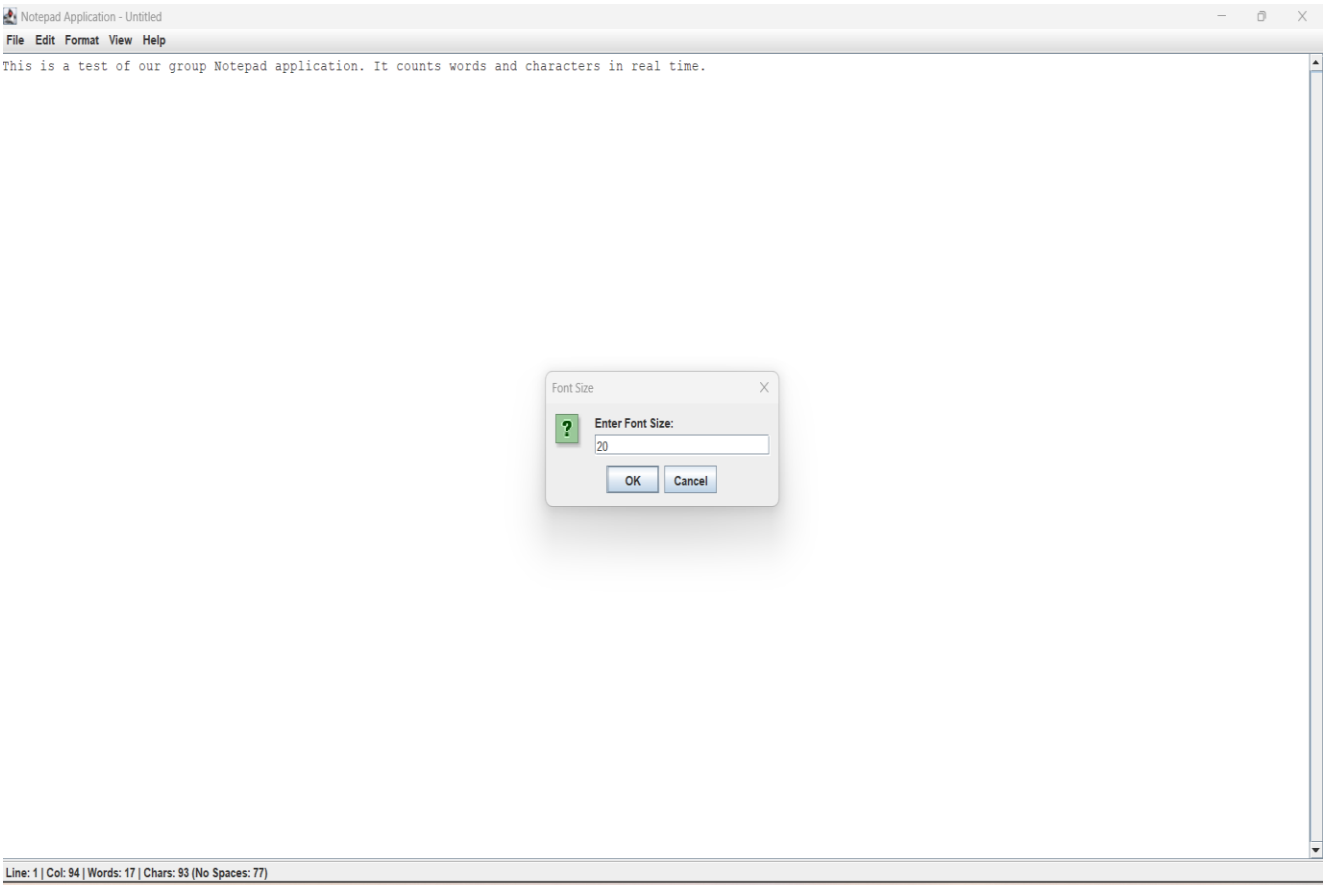


Figure 2: The Font Size Dialog

This screenshot shows the Font Size dialog opened from the Format menu. The user can type a new font size and click OK to change the size of the text. This proves that the Font Selection feature described in Module 3 is working correctly.

Module 4: Status Bar Mechanics & View Menu

- **Real time metrics (via updateStatusBar)**: will automatically calculate and display the current cursor line/column, total character count, total non-space character count and word count.
- **View Menu Toggle**: a toggleable checkbox menu item which will show/hide the status bar.

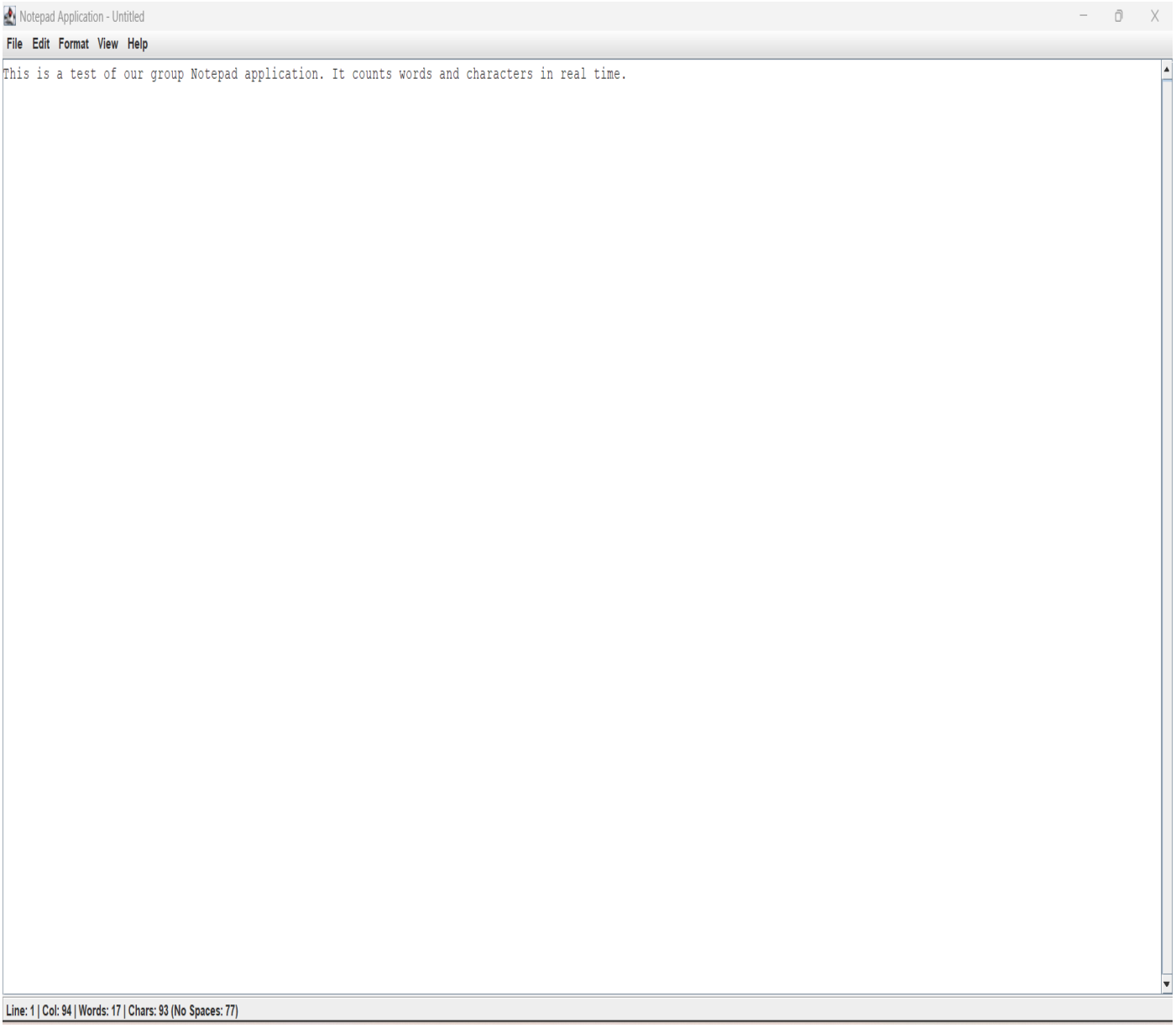


Figure 3: The Main Notepad Interface with the Status Bar

This screenshot shows the main interface of our Notepad app with some text in the text area. At the bottom is the status bar which is fully functional and displays the current line number, column number, word count and character count. All of these values are updated

in real time as you type. This proves that the Status Bar Mechanics described in Section 4 are working

Module 5: Unit Testing & Error Guards

- **Automated unit tests (NotepadLogicTest)**: JUnit test cases (testWordCountCalculation, testEmptyTextWordCount and testNonSpaceCharacterCount) to validate the text parsing and metric calculation.

3. User Guide

- **Open a file**: go to the File menu, click Open, select any .txt file from your computer and it's loaded straight into the editor.
- **Custom fonts**: go to Format to bring up the font selector dialogue, pick your preferred system font and apply it to your document.
- **Track document stats**: you can see the live character and word count right in the status bar at the bottom of the window (toggle this on and off via the View menu).